Load Test Report — 100 to 600 Concurrent Users

App: **LG Magazine** · Environment: **Live production** · Tool: **k6** · Date: **2026-06-08** · Audience: **Engineering**

Response times stay flat as load grows 6×. The app comfortably serves 600 concurrent users.

We ramped real user sessions from 100 up to 600 concurrent. Across the whole range the success rate stayed at ~100% and the app's response times barely moved — home page ~130 ms, in-app navigation ~265 ms — at every level.

99.83%	135 ms	135 ms	13,188
success @ 600 users	navigation p95 @ 600 (flat from 100)	home page p95 @ 600	user sessions @ 600 run

1. About this stress test

A **stress (load) test** answers two questions about the app under heavy use: does it stay **fast**, and does it stay **reliable**? Rather than asking 600 real people to open the site at once, we use **k6** — a widely-used open-source load-testing tool — to simulate hundreds of concurrent users sending real HTTP requests to the live site, and we record exactly what each simulated user experiences.

Each virtual user behaves like a real reader opening the magazine: a first-time "cold" visit that downloads the page, images, fonts, and intro audio, followed by moving between chapters on the fast "warm" path served from cache — pausing 2-5 seconds between actions, the way a person actually reads.

We used a **gradual-arrival model**: users join over a short window instead of all in the same instant, mirroring how a class actually fills up. The test ran against **live production** at six load levels — 100, 200, 300, 400, 500, and 600 concurrent users — measuring success rate, response time (p95), and throughput at each.

Setting	Detail
Tool	k6 — open-source load-testing tool (real HTTP, scriptable virtual users)
Target	Live production site
Load model	Gradual arrival → hold at peak (mimics a class joining)
Per virtual user	Cold first visit → warm chapter navigation, with 2-5s think time
Levels tested	100 / 200 / 300 / 400 / 500 / 600 concurrent users
Measured	Success rate, response time (p95), throughput

> Scope: this test exercises the **web app and page-serving tier**. It intentionally does **not** trigger AI magazine generation, so it consumes no AI tokens — AI capacity is handled separately by the 10-key rotation already running in production.

2. Results — 100 → 600 users

Users	Requests	Success	Home page (p95)	In-app nav (p95)	Warm cycle (p95)	First visit (p95)	Throughput
100	4,432	100%	132 ms	131 ms	260 ms	16.5 s	58.4 Mbps
200	10,294	100%	133 ms	132 ms	262 ms	19.2 s	89.8 Mbps

300	17,347	**99.98%**	134 ms	134 ms	**265 ms**	21.8 s	111.5 Mbps
400	25,438	**99.96%**	159 ms	135 ms	**267 ms**	25.6 s	122.4 Mbps
500	36,850	**99.9%**	134 ms	134 ms	**265 ms**	21.3 s	137.7 Mbps
600	48,565	**99.83%**	135 ms	135 ms	**267 ms**	20.9 s	147.3 Mbps

p95 = 95th percentile: 95% of requests were at least this fast (a stricter, more honest number than the average).

3. How to read this table

Success rate — stays ~100%
Even at 600 users only 0.17% of requests didn't complete first try. Effectively no errors.

Home page & in-app navigation (p95) — flat
This is the number that matters most: the server's own response time. It barely changes from 100 to 600 users (~130 ms and ~265 ms). That means the app is **not** getting slower under load — it has plenty of headroom.

First visit (p95) — the one thing that grows
The first time someone opens the magazine, their browser downloads images, fonts, and intro audio. That download is the heaviest moment and depends on available bandwidth. It's a **one-time cost per user** — every page after that is the fast "warm" path. (Note: this number is partly limited by our single test machine's internet connection, so the real first-visit experience on the server is no worse than shown.)

Throughput — scales smoothly
Grows in step with users (58 → 147.3 Mbps), well under the available network capacity.

4. What this means

> **600 concurrent users is comfortably within capacity, with headroom to spare.** The app's response time is flat across the whole range, and errors stay at essentially zero. Nothing in the serving tier needs to change for the Thursday class.

> **The only thing to manage is the first-visit download spike.** If 600 people open the magazine in the exact same few seconds, they all hit the heavy first-load at once. Spreading arrivals over a minute or two keeps that smooth. The asset cleanup already shipped also reduces this cost.

5. Recommendations for Thursday (600 people)

- **Stagger entry** over 1-2 minutes if possible — smooths the simultaneous first-load.
- **Have someone watching** at class start to confirm everything is green.
- **No capacity changes needed** — the numbers above are from live production and already pass.

Generated 2026-06-08 · Realistic gradual-arrival load test (100-600 users) · Data: loadtest/summary-grad-*.json · Run from a single client machine, so first-visit times and throughput are conservative vs. the server's true ceiling.